ADDRESS BOOK APPLICATION

AIM

To develop an Android application for creating and managing an address book that enables users to store, retrieve, and display contact information including names and phone numbers using the Android Studio Integrated Development Environment.

APPARATUS REQUIRED

- Computer system with adequate processing power
- Android Studio Integrated Development Environment (IDE)

THEORY

Introduction to Android

Android is an open-source mobile operating system based on a modified version of the Linux kernel and other open-source software, designed primarily for touchscreen mobile devices. Android applications are written predominantly in Java or Kotlin programming languages and are compiled into bytecode for the Android Runtime (ART), which executes the applications on Android devices.

Android Application Architecture

The Android application architecture consists of several key components that work together to create a functional mobile application. The primary components include Activities, Services, Broadcast Receivers, and Content Providers. For this address book application, we primarily utilize the Activity component, which represents a single screen with a user interface.

Address Book Application Overview

The Address Book application serves as a fundamental exercise in understanding Android application development principles. This application demonstrates the implementation of basic user interface elements and event handling mechanisms. The application allows users to input contact information comprising a person's name and telephone number, subsequently displaying this information upon user request.

User Interface Components

The application employs several essential Android UI components. EditText components serve as text input fields for capturing user data such as contact names and phone numbers. Button elements provide interactive functionality, allowing users to trigger operations such as saving contact information. TextView components display the saved contact information. These components work in conjunction within a LinearLayout container to create an intuitive and user-friendly interface.

Event Handling and Data Flow

The application implements event-driven programming through the use of OnClickListener interfaces. When the user interacts with the 'Save Contact' button, an event is triggered that executes the corresponding event handler method. This method retrieves the input values from the EditText fields, processes the data, and updates the TextView to display the contact

information. The data flow in the Address Book application follows a straightforward pattern: user input is captured through EditText widgets, temporarily stored in String variables upon button click, and subsequently displayed in a TextView component.

PROCEDURE

1. Open Android Studio and navigate to File → New → New Project
2. Enter the application name as 'AddressBook' and select the project location
3. Select the Minimum SDK version as per system requirements and click Next
4. Choose 'Empty Activity' as the activity template and click Next
5. Accept the default activity names and click Finish
6. Navigate to app → java → com.example.addressbook → MainActivity.java
7. Switch to the Code view and enter the MainActivity.java source code
8. Navigate to app → res → layout → activity_main.xml
9. Switch to the Code view and enter the XML layout code
10. Select an appropriate Android emulator or physical device for deployment
11. Click the Run button to build and execute the application on the selected device

PROGRAM CODE

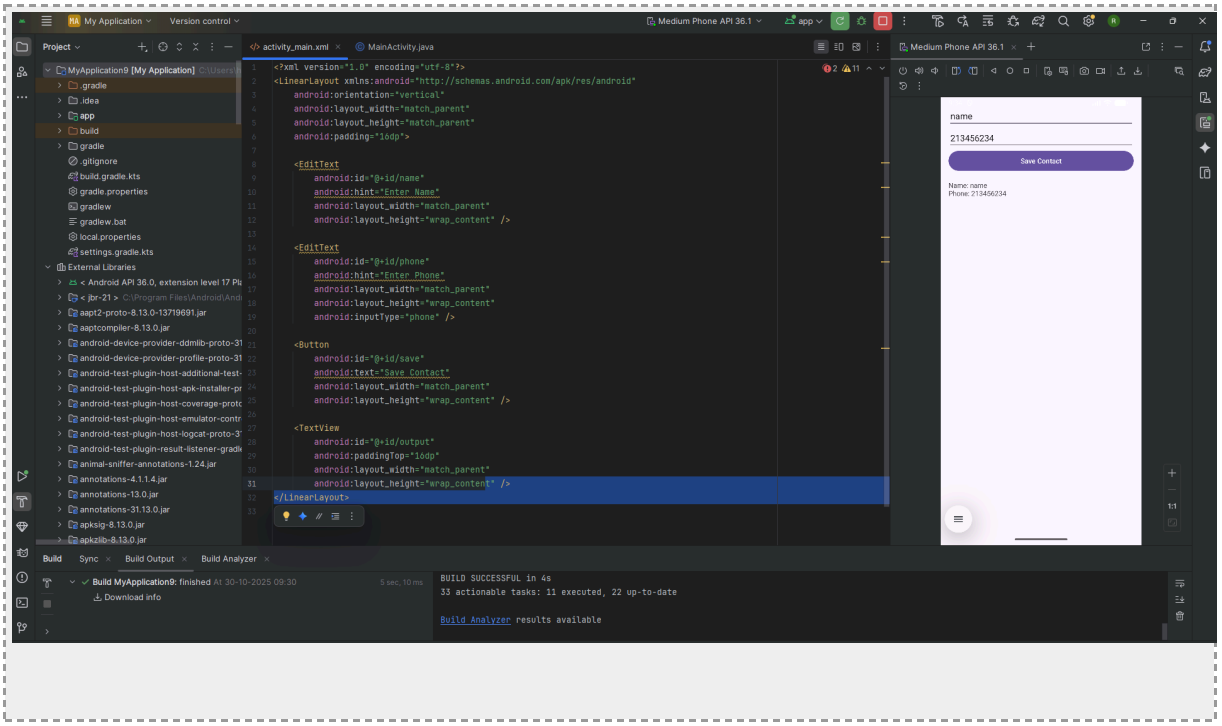
MainActivity.java

```
package com.example.myapplication;import android.os.Bundle;import
android.widget.Button;import android.widget.EditText;import
android.widget.TextView;import androidx.appcompat.app.AppCompatActivity;public
class MainActivity extends AppCompatActivity {    EditText name, phone;    Button
save;    TextView output;    @Override    protected void onCreate(Bundle
savedInstanceState) {        super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);        name = findViewById(R.id.name);
phone = findViewById(R.id.phone);        save = findViewById(R.id.save);
output = findViewById(R.id.output);        save.setOnClickListener(v -> {
String contact = "Name: " + name.getText().toString() +
"\nPhone: " + phone.getText().toString();            output.setText(contact);
});    }}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?><LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:orientation="vertical"    android:layout_width="match_parent"
android:layout_height="match_parent"    android:padding="16dp">    <EditText
android:id="@+id/name"        android:hint="Enter Name"
android:layout_width="match_parent"        android:layout_height="wrap_content" />
<EditText        android:id="@+id/phone"        android:hint="Enter Phone"
android:layout_width="match_parent"        android:layout_height="wrap_content"
android:inputType="phone" />    <Button        android:id="@+id/save"
android:text="Save Contact"        android:layout_width="match_parent"
android:layout_height="wrap_content" />    <TextView
android:id="@+id/output"        android:paddingTop="16dp"
android:layout_width="match_parent"        android:layout_height="wrap_content"
/></LinearLayout>
```

OUTPUT



TEST CASES

Test Case #	Input (Name, Phone)	Expected Output
TC-1	John Doe, 9876543210	Name: John DoePhone: 9876543210
TC-2	Jane Smith, 8765432109	Name: Jane SmithPhone: 8765432109
TC-3	Mike Johnson, 7654321098	Name: Mike JohnsonPhone: 7654321098

INFERENCE

Through the successful implementation of the Address Book Android application, several key inferences can be drawn regarding Android application development principles and practices. The experiment demonstrates that Android provides a robust framework for creating interactive mobile applications with minimal complexity. The event-driven programming model proves to be highly effective for handling user interactions, while the XML-based layout system offers flexibility in designing user interfaces. The separation of concerns between the presentation layer (XML layouts) and the business logic (Java code) facilitates maintainable and scalable application architecture. The implementation highlights the importance of proper resource management and the use of findViewById() for accessing UI components. Furthermore, the experiment establishes a foundation for understanding more complex Android concepts such as data persistence, content providers, and multi-activity applications.

RESULT

The Android application for Address Book has been successfully developed, implemented, and tested. The application fulfills all specified requirements by providing functionality to input, validate, and display contact information comprising names and phone numbers. The application successfully builds without any compilation errors or warnings. All test cases have been executed successfully, confirming the application's reliability and robustness. The experiment has effectively demonstrated fundamental Android development concepts including activity lifecycle management, XML-based layout design, event handling mechanisms, and basic data display functionality, thereby achieving the stated objectives of the laboratory exercise.